MODUL 14 · BAB V · ALGORITMA GREEDY

# Activity-Selection Problem dengan Algoritma Greedy

Menjadwalkan sebanyak mungkin aktivitas pada satu sumber daya dengan memilih aktivitas yang paling cepat selesai.

### TUJUAN PEMBELAJARAN

Setelah mempelajari modul ini, kamu diharapkan mampu:

- Merumuskan activity-selection problem, termasuk arti selang setengah terbuka dan aktivitas yang kompatibel.
- Menjelaskan mengapa kriteria "waktu selesai paling awal" selalu menghasilkan solusi optimal (argumen pertukaran).
- Menjalankan GREEDY-ACTIVITY-SELECTOR langkah demi langkah dan menentukan running time-nya.
- Menunjukkan dengan contoh mengapa kriteria waktu mulai paling awal atau durasi paling pendek tidak selalu optimal.

### PRASYARAT

- Modul 13: konsep Greedy dan greedy-choice property.
- Modul 04/08: pengurutan dalam waktu $O(n \lg n)$.

### BAB / MATERI ASAL

Materi 10.1 ALGORITMA GREEDY An Activity Selection Problem

### POSISI DALAM URUTAN BELAJAR

Penerapan pertama Greedy (Modul 13). Diikuti Huffman (Modul 15).

### DAFTAR ISI


1. Satu Ruangan, Banyak Kegiatan ..... 1
2. Rumusan Masalah ..... 2
3. Pilihan Greedy: Selesai Paling Awal ..... 3
4. Prosedur GREEDY-ACTIVITY-SELECTOR ..... 3
5. Penelusuran dan Latihan ..... 4
6. Analisis dan Keterkaitan ..... 7
7. Kesimpulan ..... 8
- Kuis Pemahaman ..... 9
- Kunci Jawaban dan Pembahasan ..... 10
- Referensi ..... 10


PENDAHULUAN

## 1. Satu Ruangan, Banyak Kegiatan

Sebuah ruang kuliah hanya dapat dipakai oleh satu kegiatan pada satu waktu. Ada sebelas kegiatan yang mengajukan pemakaian ruangan, masing-masing dengan jam mulai dan jam selesai. Beberapa kegiatan waktunya bertabrakan. Bagaimana memilih kegiatan agar **sebanyak mungkin** kegiatan dapat berlangsung tanpa ada yang bertabrakan?

Masalah ini disebut **activity-selection problem**. Ia merupakan masalah optimasi yang dapat diselesaikan dengan Dynamic Programming, tetapi juga dengan algoritma Greedy yang jauh lebih sederhana. Modul ini

membahas algoritma Greedy-nya, mengapa algoritma itu selalu benar, dan mengapa pilihan kriteria greedy sangat menentukan, sebagaimana diperlihatkan oleh latihan pada bahan kuliah.

PEMBAHASAN INTI

## 2. Rumusan Masalah

Misalkan $S = \{a_1, a_2, \ldots, a_n\}$ adalah himpunan $n$ aktivitas yang membutuhkan sumber daya yang sama. Setiap aktivitas $a_i$ memiliki **waktu mulai** (*start time*) $s_i$ dan **waktu selesai** (*finish time*) $f_i$, dengan $0 \le s_i < f_i < \infty$. Aktivitas $a_i$ yang dipilih menempati **selang setengah terbuka** $[s_i, f_i)$.

DEFINISI

Aktivitas $a_i$ dan $a_j$ **kompatibel** jika selang $[s_i, f_i)$ dan $[s_j, f_j)$ tidak tumpang tindih, yaitu jika $s_j \ge f_i$ atau $s_i \ge f_j$.

**Activity-selection problem**: pilih himpunan bagian aktivitas yang saling kompatibel dengan **ukuran maksimum**.

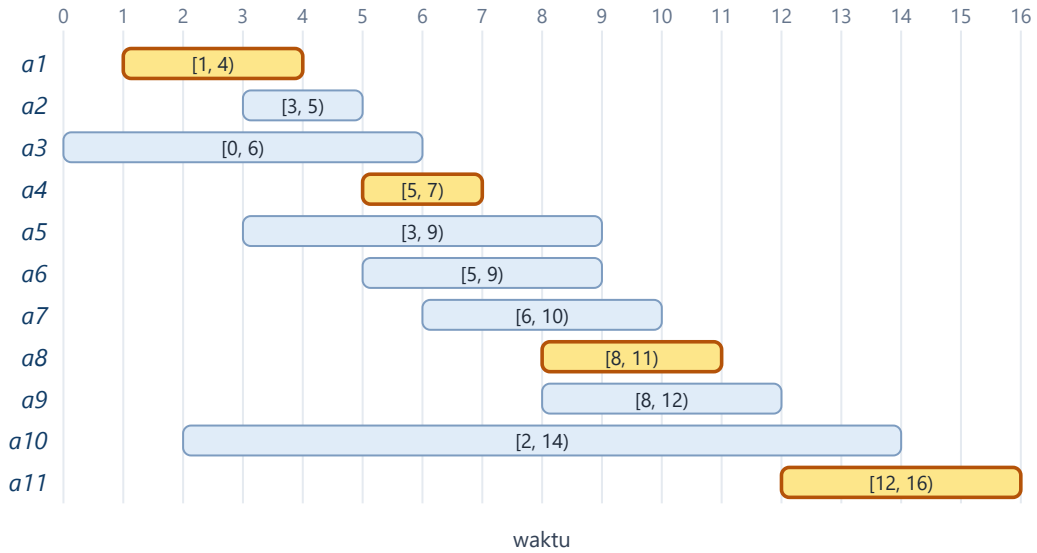
Arti "setengah terbuka": waktu selesai $f_i$ tidak termasuk dalam selang. Jadi aktivitas yang selesai pukul 5 kompatibel dengan aktivitas yang mulai tepat pukul 5. Itulah sebabnya syarat kompatibel memakai $\ge$, bukan $>$.

Pada algoritma Greedy, aktivitas diasumsikan sudah **terurut naik berdasarkan waktu selesai**: $f_1 \le f_2 \le \cdots \le f_n$.

### 2.1 Contoh dari bahan kuliah

| $i$ | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $s_i$ | 1 | 3 | 0 | 5 | 3 | 5 | 6 | 8 | 8 | 2 | 12 |
| $f_i$ | 4 | 5 | 6 | 7 | 9 | 9 | 10 | 11 | 12 | 14 | 16 |

**Tabel 1.** Himpunan 11 aktivitas, terurut berdasarkan waktu selesai.




**Gambar 1.** Aktivitas pada garis waktu. Batang kuning adalah solusi Greedy $\{a_1, a_4, a_8, a_{11}\}$.

### 2.2 Penelusuran manual

Bahan kuliah mencoba beberapa aktivitas pertama, lalu memilih aktivitas berikutnya yang waktu mulainya tidak lebih awal dari waktu selesai aktivitas sebelumnya:

- Mulai dari $a_3$ (waktu mulai 0): $\{a_3, a_7, a_{11}\}$, 3 aktivitas.
- Mulai dari $a_1$: salah satu himpunan yang dapat dipilih $\{a_1, a_4, a_8, a_{11}\}$, 4 aktivitas.
- Mulai dari $a_2$: salah satu himpunan yang dapat dipilih $\{a_2, a_4, a_8, a_{11}\}$, 4 aktivitas.

Secara manual, ukuran maksimumnya adalah **4**. (Pemeriksaan seluruh $2^{11}$ himpunan bagian dengan program mengonfirmasi bahwa tidak ada 5 aktivitas yang saling kompatibel.)

**CATATAN KOREKSI TERHADAP SUMBER**

Bahan kuliah menulis syarat "setiap start time yang akan dipilih harus **lebih besar** dari finish time aktivitas sebelumnya". Sesuai definisi kompatibel dan pseudocode (langkah 5: $s[m] \geq f[k]$), syarat yang tepat adalah **lebih besar atau sama dengan**. Perbedaan ini berpengaruh: pada himpunan $\{a_2, a_4, a_8, a_{11}\}$, aktivitas $a_4$ mulai pukul 5 tepat saat $a_2$ selesai ($s_4 = f_2 = 5$). Dengan syarat "lebih besar" yang ketat, himpunan itu justru tidak sah.

**PEMBAHASAN INTI**

## 3. Pilihan Greedy: Selesai Paling Awal

Gagasan intuitifnya: pilih aktivitas yang **paling cepat selesai**, karena aktivitas itu menyisakan waktu sebanyak mungkin untuk aktivitas-aktivitas berikutnya. Setelah memilihnya, buang semua aktivitas yang bertabrakan dengannya, lalu ulangi pada aktivitas yang tersisa.

**TEOREMA (CORMEN DKK., 2009, TEOREMA 16.1)**

Untuk setiap himpunan aktivitas tak kosong, aktivitas dengan waktu selesai paling awal termasuk dalam **suatu** himpunan aktivitas kompatibel berukuran maksimum.

**Bukti (argumen pertukaran).** Misalkan $a_1$ adalah aktivitas yang paling awal selesai, dan $A$ adalah sembarang solusi optimal. Urutkan aktivitas di $A$ menurut waktu selesai, dan misalkan $a_j$ adalah yang pertama.

- Jika $a_j = a_1$, maka $A$ sudah memuat $a_1$. Selesai.
- Jika $a_j \neq a_1$, buat $A' = (A - \{a_j\}) \cup \{a_1\}$, yaitu tukar $a_j$ dengan $a_1$. Karena $f_1 \leq f_j$, dan semua aktivitas lain di $A$ mulai setelah $a_j$ selesai ($s \geq f_j \geq f_1$), maka $a_1$ kompatibel dengan semuanya. Jadi $A'$ sah dan $|A'| = |A|$, sehingga $A'$ juga optimal dan memuat $a_1$.

Setelah $a_1$ dipilih, yang tersisa adalah submasalah yang sama bentuknya: aktivitas-aktivitas yang mulai setelah $f_1$. Dengan menerapkan argumen yang sama berulang kali, seluruh pilihan greedy menghasilkan solusi optimal. Inilah **greedy-choice property** dan **optimal substructure** dari Modul 13.

PEMBAHASAN INTI

## 4. Prosedur GREEDY-ACTIVITY-SELECTOR

Masukan: array $s$ dan $f$ berisi waktu mulai dan waktu selesai, dengan aktivitas terurut berdasarkan $f$. Setiap aktivitas yang dipilih dimasukkan ke himpunan $A$, yang menjadi hasil akhir.

GREEDY-ACTIVITY-SELECTOR(s, f)

```
1  n = s.length
2  A = {a1}
3  k = 1
4  for m = 2 to n
5      if s[m] ≥ f[k]
6          A = A ∪ {am}
7          k = m
8  return A
```

| Langkah | Penjelasan |
|---|---|
| 1 | Inisialisasi $n$ = banyaknya aktivitas. |
| 2 | Himpunan $A$ diawali dengan $a_1$. Prosedur selalu memilih $a_1$ karena $a_1$ paling awal selesai (pilihan greedy pertama). |
| 3 | $k$ menunjuk aktivitas yang terakhir dimasukkan ke $A$. Karena terurut, $f_k$ adalah waktu selesai **maksimum** di antara semua aktivitas di $A$. |
| 4–7 | Periksa setiap aktivitas $a_m$ secara berurutan. Jika $s[m] \geq f[k]$ (langkah 5), $a_m$ kompatibel dengan semua aktivitas di $A$: masukkan ke $A$ (langkah 6) dan perbarui $k = m$ (langkah 7). Jika tidak, lewati $a_m$. |
| 8 | Kembalikan $A$, himpunan aktivitas kompatibel dengan ukuran maksimum. |

**Tabel 2.** Penjelasan langkah-langkah GREEDY-ACTIVITY-SELECTOR.

Cukup membandingkan dengan $f_k$ saja (tidak perlu dengan semua anggota $A$), karena $a_k$ adalah anggota $A$ yang paling akhir selesai. Hal ini **hanya** benar bila aktivitas terurut berdasarkan waktu selesai.

CONTOH DAN PENERAPAN

# 5. Penelusuran dan Latihan

## 5.1 Penelusuran pada 11 aktivitas

| Loop | $m$ | $s[m]$ | $f[k]$ | $s[m] \geq f[k]$? | $A$ | $k$ |
|---|---|---|---|---|---|---|
| awal | - | - | - | - | $\{a_1\}$ | 1 |
| 1 | 2 | 3 | $f[1] = 4$ | False | $\{a_1\}$ | 1 |
| 2 | 3 | 0 | 4 | False | $\{a_1\}$ | 1 |
| 3 | 4 | 5 | 4 | True | $\{a_1, a_4\}$ | 4 |
| 4 | 5 | 3 | $f[4] = 7$ | False | $\{a_1, a_4\}$ | 4 |
| 5 | 6 | 5 | 7 | False | $\{a_1, a_4\}$ | 4 |
| 6 | 7 | 6 | 7 | False | $\{a_1, a_4\}$ | 4 |
| 7 | 8 | 8 | 7 | True | $\{a_1, a_4, a_8\}$ | 8 |
| 8 | 9 | 8 | $f[8] = 11$ | False | $\{a_1, a_4, a_8\}$ | 8 |
| 9 | 10 | 2 | 11 | False | $\{a_1, a_4, a_8\}$ | 8 |
| 10 | 11 | 12 | 11 | True | $\{a_1, a_4, a_8, a_{11}\}$ | 11 |

**Tabel 3.** Penelusuran GREEDY-ACTIVITY-SELECTOR.

Perulangan `for` berakhir setelah $m = 11$ (pemeriksaan $m = 12 > n$ menghentikan perulangan). Hasil: $A = \{a_1, a_4, a_8, a_{11}\}$, 4 aktivitas, sama dengan ukuran maksimum yang ditemukan secara manual.

CATATAN KOREKSI TERHADAP SUMBER

Dua salah ketik kecil pada penelusuran di bahan kuliah: (1) pada Loop 7 tertulis "$s[8] = 8$, $f[4] = 4$"; seharusnya $f[4] = 7$ (kesimpulan True tetap benar karena $8 \geq 7$). (2) Baris "Loop 10: m = 12, False" sebenarnya adalah pemeriksaan kepala perulangan yang ke-11, yaitu saat $m = 12$ melebihi $n = 11$ sehingga perulangan berhenti; tidak ada aktivitas ke-12.

## 5.2 Running time

Perulangan pada langkah 4 berjalan $n - 1$ kali, dan setiap putarannya berbiaya konstan. Jadi GREEDY-ACTIVITY-SELECTOR berjalan dalam waktu $\Theta(n)$, dengan asumsi aktivitas sudah terurut berdasarkan waktu selesai. Jika belum terurut, perlu pengurutan terlebih dahulu dalam waktu $O(n \lg n)$ (misalnya Merge Sort, Modul 08), sehingga total waktunya $O(n \lg n)$.

## 5.3 Latihan dari bahan kuliah: mengganti kriteria pengurutan

Soal: susun aktivitas terurut naik berdasarkan (a) waktu mulai $s_i$, (b) durasi $d_i = f_i - s_i$. Jalankan prosedur yang sama. Apakah hasilnya tetap optimal (4 aktivitas)?

**(a) Terurut berdasarkan waktu mulai.** Indeks baru 1..11 menunjuk urutan baru:

| $i$ | **1** | 2 | 3 | 4 | 5 | 6 | 7 | **8** | 9 | 10 | **11** |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $s_i$ | 0 | 1 | 2 | 3 | 3 | 5 | 5 | 6 | 8 | 8 | 12 |
| $f_i$ | 6 | 4 | 14 | 5 | 9 | 7 | 9 | 10 | 11 | 12 | 16 |

**Tabel 4.** Aktivitas terurut berdasarkan waktu mulai (indeks sudah diurutkan ulang).

Hasil: $A = \{a_1, a_8, a_{11}\}$ (selang $[0, 6), [6, 10), [12, 16)$), hanya **3 aktivitas, tidak optimal**. Aktivitas pertama $[0, 6)$ memang mulai paling awal, tetapi selesainya terlambat dan menghalangi banyak aktivitas pendek.

**(b) Terurut berdasarkan durasi.**

| $i$ | **1** | **2** | 3 | **4** | 5 | 6 | 7 | **8** | 9 | 10 | 11 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| $s_i$ | 3 | 5 | 1 | 8 | 5 | 6 | 8 | 12 | 0 | 3 | 2 |
| $f_i$ | 5 | 7 | 4 | 11 | 9 | 10 | 12 | 16 | 6 | 9 | 14 |
| $d_i$ | 2 | 2 | 3 | 3 | 4 | 4 | 4 | 4 | 6 | 6 | 12 |

**Tabel 5.** Aktivitas terurut berdasarkan durasi (indeks sudah diurutkan ulang).

Hasil: $A = \{a_1, a_2, a_4, a_8\}$ (selang $[3, 5), [5, 7), [8, 11), [12, 16)$), 4 aktivitas. Pada data ini hasilnya **kebetulan optimal**.

**DURASI TERPENDEK TIDAK SELALU OPTIMAL**

Hasil (b) tidak berarti kriteria durasi terpendek selalu benar. Contoh penyangkal kecil: tiga aktivitas $[1, 5)$, $[4, 7)$, $[6, 10)$. Aktivitas terpendek adalah $[4, 7)$ (durasi 3), tetapi memilihnya membuat kedua aktivitas lain tidak dapat dipilih, sehingga hasilnya hanya **1** aktivitas. Solusi optimalnya **2** aktivitas: $[1, 5)$ dan $[6, 10)$. Hanya kriteria **waktu selesai paling awal** yang terbukti selalu optimal.

Catatan teknis: pemeriksaan $s[m] \geq f[k]$ yang hanya membandingkan dengan aktivitas terakhir dipilih tidak menjamin kompatibilitas bila data tidak terurut menurut waktu selesai. Pada kedua latihan di atas hasilnya kebetulan tetap kompatibel; pada data lain, kriteria selain waktu selesai perlu memeriksa kecocokan dengan **semua** aktivitas yang sudah dipilih.

### 5.4 Implementasi Python

Python: tiga kriteria pengurutan dan contoh penyangkal

```python
def greedy_activity_selector(s, f):          # s, f sudah terurut menurut f
    A, k = [0], 0
    for m in range(1, len(s)):
        if s[m] >= f[k]:
            A.append(m)
            k = m
    return A

s = [1, 3, 0, 5, 3, 5, 6, 8, 8, 2, 12]
f = [4, 5, 6, 7, 9, 9, 10, 11, 12, 14, 16]
akt = list(zip(s, f))

for nama, kunci in (("selesai", lambda a: a[1]),
                    ("mulai", lambda a: a[0]),
                    ("durasi", lambda a: a[1] - a[0])):
    urut = sorted(akt, key=kunci)           # sorted stabil
    A = greedy_activity_selector([a[0] for a in urut], [a[1] for a in urut])
    print(nama, [urut[i] for i in A])

def durasi_terpendek(akt):                   # pilih yang terpendek & cocok dg semua
    dipilih = []
    for a in sorted(akt, key=lambda a: a[1] - a[0]):
        if all(a[0] >= b[1] or b[0] >= a[1] for b in dipilih):
            dipilih.append(a)
    return dipilih

print(durasi_terpendek([(1, 5), (4, 7), (6, 10)]))
```

Keluaran program

```
selesai [(1, 4), (5, 7), (8, 11), (12, 16)]
mulai [(0, 6), (6, 10), (12, 16)]
durasi [(3, 5), (5, 7), (8, 11), (12, 16)]
[(4, 7)]
```

ANALISIS DAN HUBUNGAN KONSEP

## 6. Analisis dan Keterkaitan

**Greedy vs Dynamic Programming.** Activity selection juga dapat diselesaikan dengan Dynamic Programming: definisikan $c[i, j]$ sebagai ukuran himpunan optimal untuk aktivitas yang berada di antara $a_i$ dan $a_j$, lalu coba setiap aktivitas di antaranya sebagai pilihan (Cormen dkk., 2009, Bab 16.1). Cara itu membutuhkan tabel dua dimensi dan waktu polinomial yang jauh lebih besar dari $\Theta(n)$. Greedy-choice property membuat semua pilihan itu tidak perlu dicoba: cukup satu pilihan, yaitu yang selesai paling awal.

**Kriteria greedy harus dibuktikan.** Latihan bahan kuliah memperlihatkan inti Modul 13: kriteria yang terdengar masuk akal (mulai paling awal, durasi terpendek) bisa gagal. Kriteria selesai paling awal dibuktikan benar dengan argumen pertukaran.

**Penerapan.** Pola yang sama muncul pada penjadwalan ruangan, penjadwalan mesin tunggal, dan pemilihan sinyal yang tidak saling mengganggu. Kleinberg dan Tardos (2006) menyebut masalah ini *interval scheduling* dan membuktikan kebenarannya dengan argumen "greedy stays ahead".

KESIMPULAN

## 7. Kesimpulan

- Activity-selection problem: pilih aktivitas sebanyak mungkin yang saling kompatibel; $a_i$ dan $a_j$ kompatibel jika $s_j \geq f_i$ atau $s_i \geq f_j$.
- Pilihan greedy yang benar: aktivitas dengan waktu selesai paling awal. Kebenarannya dibuktikan dengan argumen pertukaran.
- GREEDY-ACTIVITY-SELECTOR memilih $a_1$, lalu setiap $a_m$ dengan $s[m] \geq f[k]$; running time $\Theta(n)$ bila terurut, $O(n \lg n)$ termasuk pengurutan.
- Untuk 11 aktivitas pada bahan kuliah, hasilnya $\{a_1, a_4, a_8, a_{11}\}$ (4 aktivitas, optimal).
- Kriteria waktu mulai paling awal menghasilkan 3 aktivitas (tidak optimal); kriteria durasi terpendek kebetulan optimal pada data itu tetapi gagal pada contoh lain.

# Kuis Pemahaman

Kerjakan semua soal terlebih dahulu tanpa melihat kunci jawaban. Kunci jawaban beserta pembahasannya ada di halaman berikutnya.

**1 PILIHAN GANDA**

Aktivitas $a_p = [2, 6)$ dan $a_q = [6, 9)$ ...

A. Tidak kompatibel karena sama-sama memakai pukul 6.
B. Kompatibel karena $s_q = 6 \geq f_p = 6$.
C. Kompatibel hanya jika $a_q$ dipilih lebih dulu.
D. Tidak dapat ditentukan.

**2 PENELUSURAN**

Jalankan GREEDY-ACTIVITY-SELECTOR pada aktivitas berikut (sudah terurut menurut $f$): $[1, 3), [2, 5), [4, 7), [1, 8), [5, 9), [8, 10), [9, 11), [11, 14), [13, 16)$. Tuliskan himpunan $A$ yang dihasilkan.

**3 ANALISIS**

Mengapa pada langkah 5 cukup membandingkan $s[m]$ dengan $f[k]$, tidak perlu dengan semua aktivitas di $A$?

**4 BENAR/SALAH DENGAN ALASAN**

"Kriteria memilih aktivitas dengan waktu mulai paling awal selalu menghasilkan solusi optimal karena aktivitas itu tidak didahului aktivitas lain." Benar atau salah?

**5 PERHITUNGAN**

Sebanyak 10.000 aktivitas diberikan dalam keadaan acak (belum terurut). Berapa running time total untuk mendapatkan solusi optimal dengan Greedy? Bagian mana yang dominan?

**6 ANALISIS KASUS**

Buat contoh tiga atau empat aktivitas yang menunjukkan kriteria "pilih aktivitas yang paling sedikit bertabrakan dengan aktivitas lain" bisa gagal, **atau** jelaskan mengapa contoh penyangkal untuk kriteria "mulai paling awal" mudah dibuat.

**7 PEMBUKTIAN**

Dalam argumen pertukaran, mengapa $A' = (A - \{a_j\}) \cup \{a_1\}$ tetap merupakan himpunan aktivitas yang saling kompatibel?

## Kunci Jawaban dan Pembahasan

Bandingkan jawabanmu, lalu baca pembahasannya untuk memahami alasan di balik setiap jawaban.

**Soal 1**

**Jawaban: B.** Selang setengah terbuka $[2, 6)$ tidak memuat titik 6, sehingga $a_q$ boleh mulai tepat pukul 6.

**Soal 2**

$A = \{[1, 3)\}$, $f_k = 3$. $[2, 5)$: $2 \geq 3$? tidak. $[4, 7)$: $4 \geq 3$ ya, $f_k = 7$. $[1, 8)$: tidak. $[5, 9)$: tidak. $[8, 10)$: $8 \geq 7$ ya, $f_k = 10$. $[9, 11)$: tidak. $[11, 14)$: $11 \geq 10$ ya, $f_k = 14$. $[13, 16)$: tidak. Hasil: $\{[1, 3), [4, 7), [8, 10), [11, 14)\}$, 4 aktivitas.

**Soal 3**

Karena aktivitas diproses terurut menurut waktu selesai, $a_k$ (yang terakhir dimasukkan) adalah anggota $A$ dengan waktu selesai terbesar. Jika $s[m] \geq f[k]$, maka $s[m]$ juga lebih besar atau sama dengan waktu selesai semua anggota $A$ lainnya, sehingga $a_m$ kompatibel dengan semuanya.

**Soal 4**

**Salah.** Pada data bahan kuliah, kriteria ini memilih $[0, 6)$ lebih dulu dan hanya menghasilkan 3 aktivitas, padahal optimalnya 4. Aktivitas yang mulai paling awal bisa saja sangat panjang dan menghalangi banyak aktivitas lain.

**Soal 5**

Pengurutan $O(n \lg n)$ ditambah GREEDY-ACTIVITY-SELECTOR $\Theta(n)$, total $O(n \lg n)$. Bagian yang dominan adalah **pengurutan**. Untuk $n = 10.000$: sekitar $n \lg n \approx 133.000$ langkah dasar untuk mengurutkan, dibanding sekitar $10.000$ langkah untuk seleksi.

**Soal 6**

Contoh untuk kriteria "mulai paling awal": $[0, 10)$, $[1, 2)$, $[3, 4)$. Kriteria ini memilih $[0, 10)$ dan berhenti dengan 1 aktivitas, padahal $[1, 2)$ dan $[3, 4)$ (2 aktivitas) kompatibel. Contoh mudah dibuat karena cukup satu aktivitas panjang yang mulai paling awal. (Untuk kriteria "tabrakan paling sedikit", Kleinberg dan Tardos (2006) juga memberikan contoh penyangkal, tetapi contohnya lebih besar.)

**Soal 7**

Semua aktivitas lain di $A$ (selain $a_j$) mulai setelah $a_j$ selesai, yaitu $s \geq f_j$. Karena $a_1$ selesai paling awal di antara semua aktivitas, $f_1 \leq f_j \leq s$. Jadi $a_1$ selesai sebelum (atau tepat saat) setiap aktivitas lain di $A'$ dimulai, sehingga semuanya kompatibel.

## Referensi

**Sumber utama:** Materi 10.1 ALGORITMA GREEDY An Activity Selection Problem (bahan kuliah Perancangan dan Analisis Algoritma). Penjelasan tambahan, koreksi, dan pengayaan mengacu pada pustaka berikut.

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). Cambridge, MA: MIT Press. ISBN 978-0-262-03384-8.
2. Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Boston: Pearson/Addison-Wesley.